

DBMS-Project Report

UE23CS351A

Title: Food Delivery Website

Pranav Rajesh Narayan - PES1UG23CS435

Pranshu Malhotra – PES1UG23CS443

SECTION : H

Github repo link - <https://github.com/pranav-rn/food-delivery-website>

ABSTRACT-

This Food Delivery Management System is a comprehensive web-based application that facilitates online food ordering and delivery services. The system connects customers with restaurants, enabling them to browse menus, place orders, track deliveries, and manage their profiles. Built using Node.js, Express.js, and MySQL, the application provides a seamless experience for users to order food from multiple restaurants while allowing restaurant owners to manage their menus and orders efficiently. The system implements a complete order lifecycle from browsing to delivery, with features like cart management, real-time order tracking, driver assignment, and step by step delivery simulation.

User Requirement Specification-

Purpose of the Project

The Food Delivery Management System aims to digitize and streamline the food ordering and delivery process. In today's fast-paced world, customers seek convenient ways to order food from their favourite restaurants without physically visiting them. This system bridges the gap between restaurants and customers by providing a digital platform where users can browse multiple restaurant menus, customize their orders, make secure payments, and track

their deliveries in real-time. The platform also empowers restaurant owners with tools to manage their digital presence, update menus, process orders efficiently, and analyze their business performance through comprehensive reporting features. Delivery drivers too can track their weekly and monthly income and track the orders they have delivered via the driver dashboard and reports section.

Scope of the Project

The system encompasses end-to-end food delivery operations including user registration and authentication, restaurant management, menu browsing, cart operations, order placement and tracking, and delivery management. The project covers four main user roles: customers who can browse and order food, restaurant owners who can manage their establishments and menus, drivers who can see which orders are assigned to them and hence carry out the deliver, and administrators who oversee the entire platform. The system provides features for order history tracking, rating and reviews, delivery address management, and real-time order status updates. It excludes real-time GPS tracking of delivery personnel and integration with third-party payment gateways (simulated in this version). The scope is limited to web-based access.

Detailed Description

The Food Delivery Management System is a multi-user platform designed to handle the complete workflow of online food ordering and delivery.

User Management: The system supports four types of users - customers, restaurant owners, delivery drivers and administrators. Each user must register with personal details including name, email, phone number, and password. The authentication system ensures secure access with encrypted passwords. Customers can maintain multiple delivery addresses for convenience.

Restaurant Management: Restaurant owners can register their establishments with detailed information including name, description, cuisine type, operating hours, and location. They have full control over their menu items, including adding, updating, and removing dishes with details like name, description, price, category, and availability status. Restaurants can view all orders placed at their establishment and update order statuses.

Menu and Ordering System: Customers can browse restaurants based on various criteria and view detailed menus with item descriptions and prices. The

shopping cart functionality allows users to add multiple items, specify quantities, and modify their selections before checkout. The system calculates totals including applicable taxes and delivery charges.

Delivery Driver Management: Delivery drivers can view the orders they receive to pick up and can update the status of the delivery from waiting to prepared to out for delivery in real time which reflects in customers order status. They can also view their past orders and delivery history via the driver dashboard and also view their progress and income via the driver reports section.

Order Processing: When a customer places an order, the system creates an order record with a unique ID, timestamps, delivery address, and payment details. Orders go through various states: pending, confirmed, preparing, out for delivery, and delivered. Each state transition is tracked in the database. Order items are stored separately, maintaining references to the menu items and quantities ordered.

Payment System: The system simulates payment processing with support for multiple payment methods (credit card, debit card, UPI, cash on delivery). Payment records are maintained with transaction IDs and status tracking.

Review and Rating System: After order delivery, customers can rate restaurants and provide feedback, helping other users make informed decisions and restaurants improve their services.

Reporting and Analytics: The system generates various reports for restaurants, delivery drivers and administrators, including sales reports, popular items, customer ordering patterns, income, and delivery performance metrics.

Functional Requirements-

1. User Registration and Authentication
 - Allows new users to create accounts with email verification and enables secure login/logout functionality with password encryption
2. Restaurant Registration and Management
 - Enables restaurant owners to register their establishments and manage complete restaurant information including operating hours and contact details
3. Menu Management
 - Provides restaurant owners ability to add, update, delete, and categorize menu items with prices, descriptions, and images

4. Restaurant Search and Filtering
 - Allows customers to search restaurants by name, cuisine type, location, and ratings with sorting options
5. Shopping Cart Operations
 - Enables customers to add/remove items, update quantities, and view cart total before checkout
6. Order Placement and Processing
 - Facilitates complete order creation with item selection, delivery address, payment method, and order confirmation
7. Order Status Tracking
 - Provides real-time order status updates from placement to delivery with notification system
8. Delivery Address Management
 - Allows customers to save multiple delivery addresses and set default addresses for quick checkout
9. Payment Processing
 - Simulates secure payment processing with multiple payment method support and transaction record maintenance
10. Order History and Reordering
 - Maintains complete order history for customers with ability to reorder previous orders
11. Rating and Review System
 - Enables customers to rate restaurants and delivery experience with written reviews after order completion
12. Restaurant Dashboard
 - Provides restaurant owners with interface to view incoming orders, manage order status, and view analytics
13. Sales Reports and Analytics
 - Generates comprehensive reports on sales, popular items, peak ordering times, and revenue analysis
14. User Profile Management
 - Allows users to update personal information, change passwords, and manage account settings

List of Software/Tools/Programming Languages Used-

Backend Technologies:

- Node.js (Runtime Environment)
- Express.js (Web Application Framework)
- MySQL (Database Management System)

Frontend Technologies:

- HTML5
- CSS3
- JavaScript (Vanilla)
- EJS (Embedded JavaScript Templates)

Development Tools:

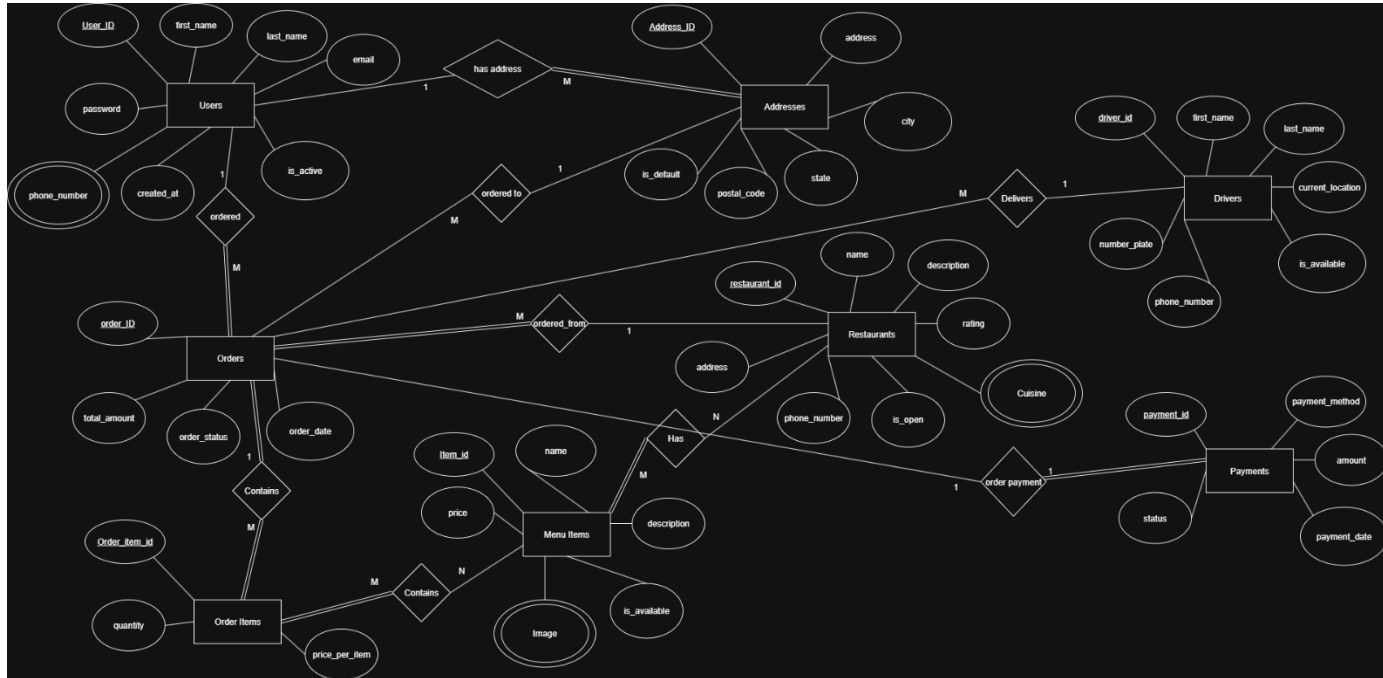
- Visual Studio Code (IDE)
- MySQL Workbench (Database Design and Management)
- Git (Version Control)
- npm (Package Manager)

Key npm Packages:

- express
- mysql2
- body-parser
- express-session
- bcryptjs (for password hashing)
- dotenv (environment variables management)

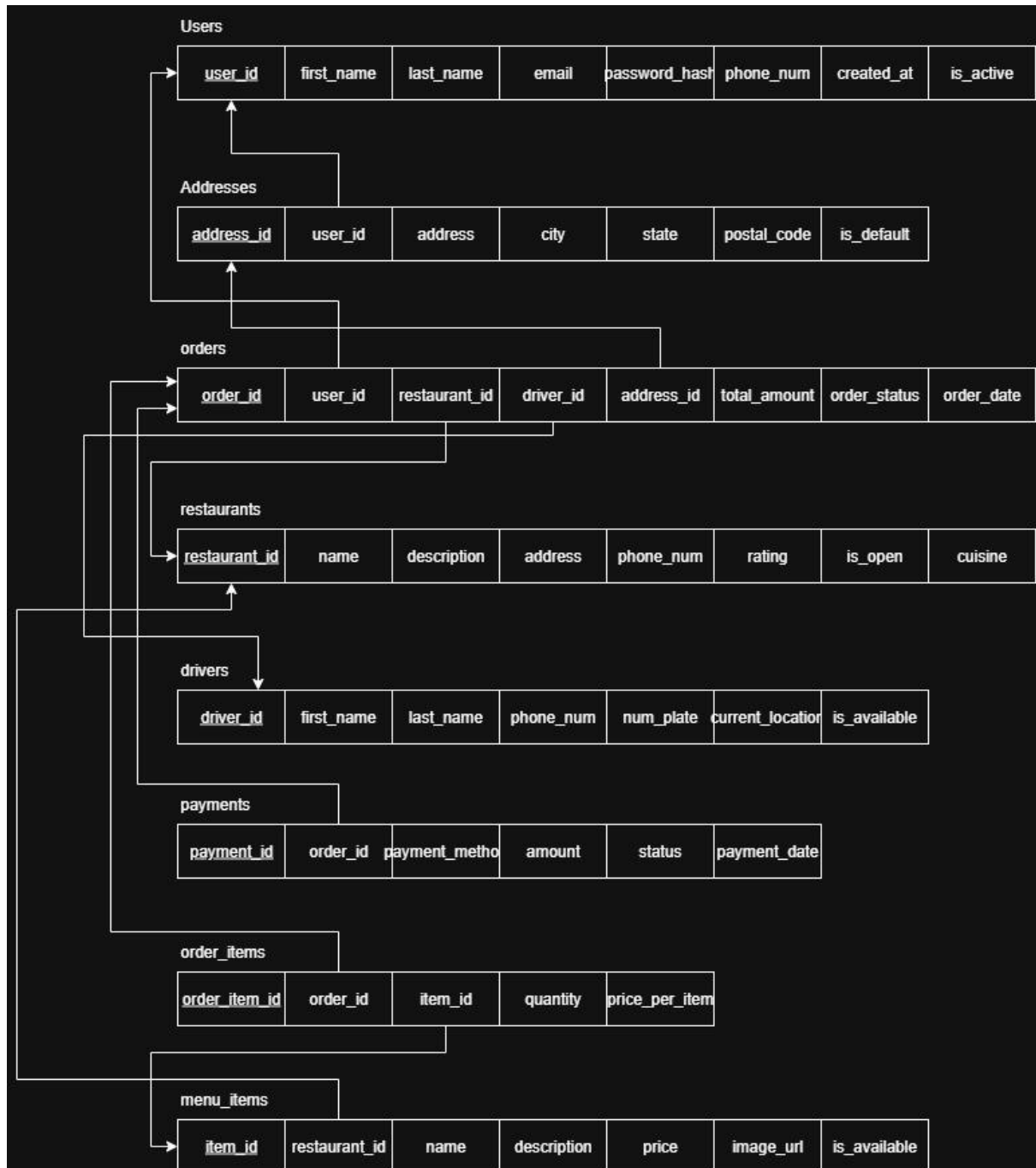
ER Diagram-

[https://github.com/pranav-rn/food-delivery-website/blob/main/Project ER diagram.png](https://github.com/pranav-rn/food-delivery-website/blob/main/Project%20ER%20diagram.png)



Relational Schema-

https://github.com/pranav-rn/food-delivery-website/blob/main/project_relational_schema.jpg



DDL COMMANDS-

https://github.com/pranav-rn/food-delivery-website/blob/main/1_database_setup.sql

-- Users Table

```
CREATE TABLE Users (  
    user_id INT AUTO_INCREMENT PRIMARY KEY,  
    first_name VARCHAR(100) NOT NULL,  
    last_name VARCHAR(100) NOT NULL,  
    email VARCHAR(150) UNIQUE NOT NULL,  
    password_hash VARCHAR(255) NOT NULL,  
    phone_num BIGINT,  
    user_type ENUM('customer', 'driver', 'restaurant_owner') DEFAULT  
'customer',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    is_active BOOLEAN DEFAULT TRUE  
);
```

-- Addresses Table

```
CREATE TABLE Addresses (  
    address_id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT NOT NULL,  
    address VARCHAR(255) NOT NULL,  
    city VARCHAR(100) NOT NULL,  
    state VARCHAR(100) NOT NULL,  
    postal_code VARCHAR(20) NOT NULL,  
    latitude DECIMAL(10, 8) DEFAULT NULL,  
    longitude DECIMAL(11, 8) DEFAULT NULL,
```



```
is_default BOOLEAN DEFAULT FALSE,  
FOREIGN KEY (user_id) REFERENCES Users(user_id)  
);
```

-- Restaurants Table

```
CREATE TABLE Restaurants (  
    restaurant_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(150) NOT NULL,  
    description TEXT,  
    address VARCHAR(255) NOT NULL,  
    latitude DECIMAL(10, 8) DEFAULT NULL,  
    longitude DECIMAL(11, 8) DEFAULT NULL,  
    phone_num VARCHAR(20),  
    rating DECIMAL(2,1) DEFAULT 0.0,  
    is_open BOOLEAN DEFAULT TRUE,  
    cuisine VARCHAR(100),  
    image LONGBLOB  
);
```

-- Restaurant_Owners Table (links users to restaurants they own)

```
CREATE TABLE Restaurant_Owners (  
    owner_id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT NOT NULL UNIQUE,  
    restaurant_id INT NOT NULL,  
    FOREIGN KEY (user_id) REFERENCES Users(user_id),  
    FOREIGN KEY (restaurant_id) REFERENCES Restaurants(restaurant_id)
```

```
);
```

```
-- Drivers Table
```

```
CREATE TABLE Drivers (  
    driver_id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT NOT NULL UNIQUE,  
    first_name VARCHAR(100) NOT NULL,  
    last_name VARCHAR(100) NOT NULL,  
    phone_num VARCHAR(20),  
    num_plate VARCHAR(20) UNIQUE,  
    current_location VARCHAR(255),  
    current_latitude DECIMAL(10, 8) DEFAULT NULL,  
    current_longitude DECIMAL(11, 8) DEFAULT NULL,  
    is_available BOOLEAN DEFAULT TRUE,  
    FOREIGN KEY (user_id) REFERENCES Users(user_id)  
);
```

```
-- Orders Table
```

```
CREATE TABLE Orders (  
    order_id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT NOT NULL,  
    restaurant_id INT NOT NULL,  
    driver_id INT,  
    address_id INT NOT NULL,  
    total_amount DECIMAL(10,2) NOT NULL,  
    order_status VARCHAR(50) DEFAULT 'Pending',
```

```
order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
rating INT DEFAULT NULL CHECK (rating >= 1 AND rating <= 5),  
FOREIGN KEY (user_id) REFERENCES Users(user_id),  
FOREIGN KEY (restaurant_id) REFERENCES Restaurants(restaurant_id),  
FOREIGN KEY (driver_id) REFERENCES Drivers(driver_id),  
FOREIGN KEY (address_id) REFERENCES Addresses(address_id)  
);
```

-- Payments Table

```
CREATE TABLE Payments (  
    payment_id INT AUTO_INCREMENT PRIMARY KEY,  
    order_id INT NOT NULL,  
    payment_method VARCHAR(50) NOT NULL,  
    amount DECIMAL(10,2) NOT NULL,  
    status VARCHAR(50) DEFAULT 'Pending',  
    payment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (order_id) REFERENCES Orders(order_id)  
);
```

-- Menu Items Table

```
CREATE TABLE Menu_Items (  
    item_id INT AUTO_INCREMENT PRIMARY KEY,  
    restaurant_id INT NOT NULL,  
    name VARCHAR(150) NOT NULL,  
    description TEXT,  
    price DECIMAL(10,2) NOT NULL,
```

```
image LONGBLOB,  
is_available BOOLEAN DEFAULT TRUE,  
FOREIGN KEY (restaurant_id) REFERENCES Restaurants(restaurant_id)  
);
```

-- Order Items Table

```
CREATE TABLE Order_Items (  
    order_item_id INT AUTO_INCREMENT PRIMARY KEY,  
    order_id INT NOT NULL,  
    item_id INT NOT NULL,  
    quantity INT NOT NULL,  
    price_per_item DECIMAL(10,2) NOT NULL,  
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),  
    FOREIGN KEY (item_id) REFERENCES Menu_Items(item_id)  
);
```

CRUD operation Screenshots and List of functionalities/features of the application and its associated screenshots using front end-

Register page- (Create)

 Restaurants [Login](#) [Sign Up](#)

Create Your Account

First Name

Last Name

john

doe

Email

johndoe@gmail.com

Phone Number

9876543210

I am registering as

Customer

Password

Confirm Password

Register

Cart page- (Create or delete items from cart or update item quantity)

 Restaurants [My Orders](#) [Cart 2](#) [Profile](#) [Logout](#)

Your Cart

From: Pizza Place

Pepperoni Pizza

₹399.00

-

1

+

₹399.00



Margherita Pizza

₹299.00

-

1

+

₹299.00



Order Summary

Subtotal	₹698.00
Delivery Fee	₹2.99
Tax (5%)	₹34.90
Total	₹735.89

[Proceed to Checkout](#)[Clear Cart](#)

Checkout page- (Create)

FoodDelivery

Restaurants

My Orders

Cart

Profile

Login

Delivery Address

123 Main St, Apt 4B
Mumbai, Maharashtra - 400001

Default

456 Park Ave
Mumbai, Maharashtra - 400002

0

Order Summary

Subtotal₹698.00

Delivery Fee₹2.99

Tax (5%)₹34.90

Total₹735.89

Place Order

Payment Method

UPI

Credit/Debit Card

Cash on Delivery

Digital Wallet

Order Items

Pepperoni Pizza x 1₹399.00

Margherita Pizza x 1₹299.00

Restaurant owner dashboard- (Update and read)

 FoodDelivery

RestaurantsManage RestaurantProfileLogout

Restaurant Owner Dashboard

Edit Restaurant Profile

Restaurant Name:

Phone Number:

Cuisine Type:

Italian

Restaurant Image:

Choose FileNo file chosen

Description:

Best pizzas in town with authentic Italian recipes

Address:

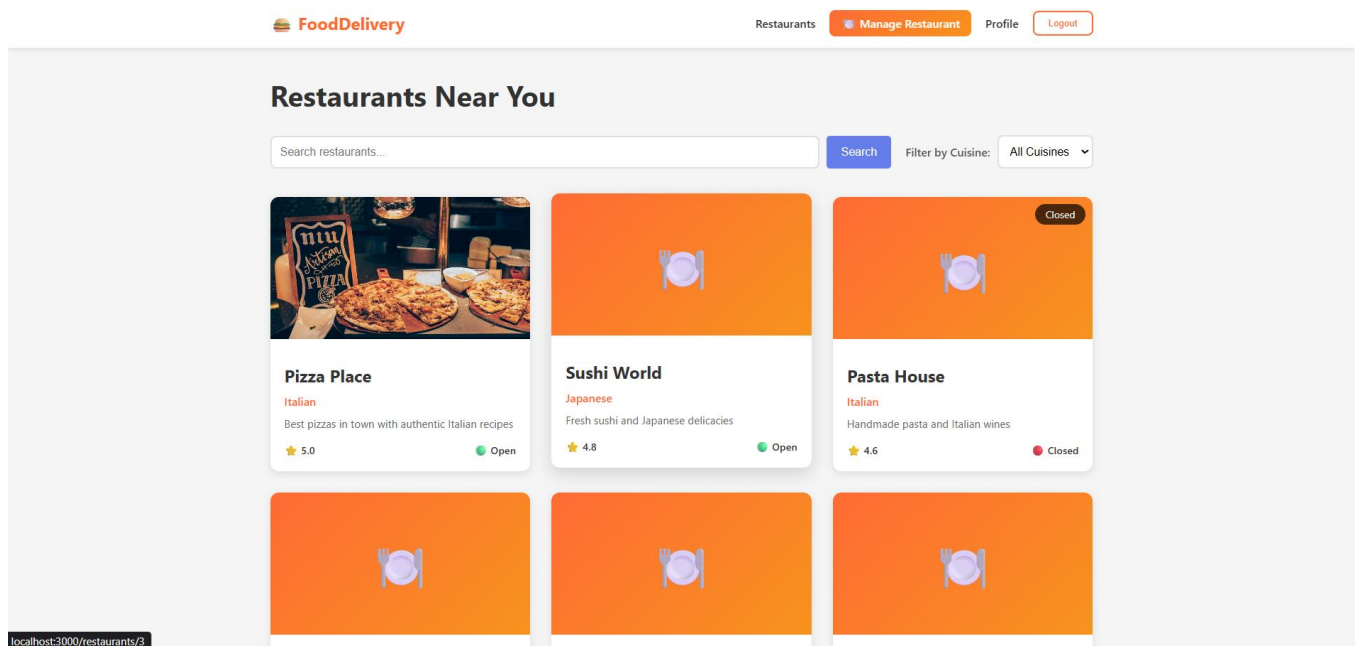
100 Food St, Mumbai



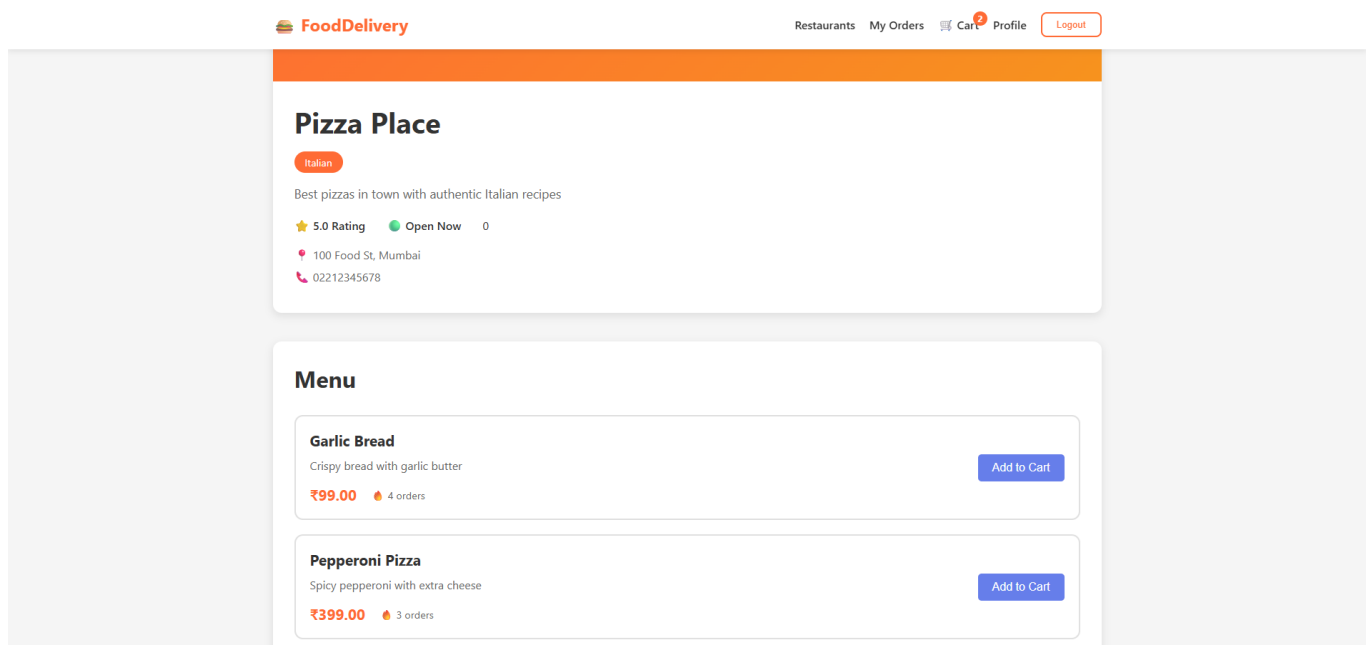
Save Changes

Cancel

Restaurants selection- (Read)



Restaurant items- (read)



Driver dashboard- (read)

RestaurantsDriver DashboardProfileLogout

Driver Dashboard

Raj Kumar

Vehicle: MH01AB1234

📍 9123456789

📍 Andheri, Mumbai

Available for Deliveries

Available Orders (0)

My Deliveries (5)

Earnings

My Deliveries

Orders assigned to you

Order #12

DELIVERED

₹99.00

Pickup

Pizza Place

100 Food St, Mumbai

📞 02212345678

Delivery

John Doe

Mumbai, Maharashtra -

📞 9876543210

Ordered: 10/31/2025, 3:44:02 PM

Driver earnings report- (read)

RestaurantsDriver DashboardProfileLogout

Driver Dashboard

Raj Kumar

Vehicle: MH01AB1234

📍 9123456789

📍 Andheri, Mumbai

Available for Deliveries

Available Orders (0)

My Deliveries (5)

Earnings

Your Earnings

TOTAL DELIVERIES

5

TOTAL EARNINGS

₹358.60

AVG PER DELIVERY

₹71.72

Daily Breakdown

Date	Deliveries	Earnings
10/31/2025	3	₹179.40
10/30/2025	1	₹19.80
10/29/2025	1	₹159.40

Restaurant items management- (Create or update or delete items)

FoodDelivery

RestaurantsManage RestaurantProfileLogout

Cancel

Edit Menu Item

Item Name *

Garlic Bread

Price (₹) *

99.00

Description

Crispy bread with garlic butter

Category *

Appetizer

Image

Choose FileNo file chosen

☐ Vegetarian

☒ Available

Update Item

Cancel

Garlic Bread

Crispy bread with garlic butter

Available

₹99.00

Edit

Delete

Margherita Pizza

Classic tomato and mozzarella

Available

₹299.00

Edit

Delete

Pepperoni Pizza

Spicy pepperoni with extra cheese

Available

₹399.00

Edit

Delete

Veggie Supreme

Loaded with fresh vegetables

Available

₹349.00

Edit

Delete

Restaurant sales analytics- (read)

FoodDelivery

RestaurantsManage RestaurantProfileLogout

Orders ReceivedManage MenuSales Analytics

Sales Overview (Last 30 Days)

TOTAL ORDERS

7

TOTAL REVENUE

₹2590.00

AVG ORDER VALUE

₹370.00

AVG RATING

5.0

★

Daily Sales

Date	Orders	Revenue	Avg Order
10/31/2025	4	₹1246.00	₹311.50
10/30/2025	1	₹99.00	₹99.00
10/29/2025	2	₹1245.00	₹622.50

Top Selling Items

Item Name	Times Ordered	Total Quantity	Revenue
Garlic Bread	4	5	₹495.00
Pepperoni Pizza	3	3	₹1197.00
Margherita Pizza	1	2	₹598.00
Veggie Supreme	1	1	₹349.00

Orders table-

```
mysql> select * from orders;
```

order_id	user_id	restaurant_id	driver_id	address_id	total_amount	order_status	order_date	rating
1	1	1	1	1	797.00	Delivered	2025-10-29 23:56:04	5
2	1	2	2	1	567.00	Delivered	2025-10-29 23:56:04	4
3	2	3	3	3	1450.00	Delivered	2025-10-29 23:56:04	5
4	3	4	1	4	797.00	Out for Delivery	2025-10-29 23:56:04	NULL
5	4	5	2	5	607.00	Preparing	2025-10-29 23:56:04	NULL
6	5	1	NULL	6	448.00	Pending	2025-10-29 23:56:04	NULL
7	1	3	4	1	1000.00	Confirmed	2025-10-29 23:56:04	NULL
8	1	1	1	1	99.00	delivered	2025-10-30 00:27:32	NULL
9	1	1	1	1	349.00	pending	2025-10-31 09:37:15	NULL
10	1	1	1	1	399.00	delivered	2025-10-31 13:59:50	5
11	1	1	1	1	399.00	delivered	2025-10-31 14:40:00	NULL
12	1	1	1	1	99.00	delivered	2025-10-31 15:44:02	5

Payments table-

```
mysql> select * from payments;
```

payment_id	order_id	payment_method	amount	status	payment_date
1	1	UPI	797.00	Completed	2025-10-29 23:56:04
2	2	Card	567.00	Completed	2025-10-29 23:56:04
3	3	Cash	1450.00	Completed	2025-10-29 23:56:04
4	4	UPI	797.00	Completed	2025-10-29 23:56:04
5	5	Card	607.00	Pending	2025-10-29 23:56:04
6	6	UPI	448.00	Pending	2025-10-29 23:56:04
7	7	Card	1000.00	Completed	2025-10-29 23:56:04
8	8	cash	106.94	completed	2025-10-30 00:27:34
9	9	cash	369.44	completed	2025-10-31 09:37:17
10	10	card	421.94	completed	2025-10-31 13:59:52
11	11	card	421.94	completed	2025-10-31 14:40:02
12	12	cash	106.94	completed	2025-10-31 15:44:04

12 rows in set (0.00 sec)

Users table-

```
mysql> select * from users;
```

user_id	first_name	last_name	email	password_hash	phone_num	user_type	created_at	is_active
1	John	Doe	john.doe@example.com	customer_pass123	9876543210	customer	2025-10-29 23:56:04	1
2	Jane	Smith	jane.smith@example.com	customer_pass123	9876543211	customer	2025-10-29 23:56:04	1
3	Alice	Johnson	alice.johnson@example.com	customer_pass123	9876543212	customer	2025-10-29 23:56:04	1
4	Bob	Williams	bob.williams@example.com	customer_pass123	9876543213	customer	2025-10-29 23:56:04	1
5	Charlie	Brown	charlie.brown@example.com	customer_pass123	9876543214	customer	2025-10-29 23:56:04	1
6	Raj	Kumar	raj.kumar@driver.com	driver_pass123	9123456789	driver	2025-10-29 23:56:04	1
7	Priya	Sharma	priya.sharma@driver.com	driver_pass123	9123456790	driver	2025-10-29 23:56:04	1
8	Amit	Patel	amit.patel@driver.com	driver_pass123	9123456791	driver	2025-10-29 23:56:04	1
9	Sunita	Gupta	sunita.gupta@driver.com	driver_pass123	9123456792	driver	2025-10-29 23:56:04	1
10	Vikram	Singh	vikram.singh@driver.com	driver_pass123	9123456793	driver	2025-10-29 23:56:04	1
11	Mario	Rossi	mario@pizzaparadise.com	owner_pass123	9111111111	restaurant_owner	2025-10-29 23:56:04	1
12	David	Chen	david@burgerhub.com	owner_pass123	9222222222	restaurant_owner	2025-10-29 23:56:04	1
13	Yuki	Tanaka	yuki@sushiworld.com	owner_pass123	9333333333	restaurant_owner	2025-10-29 23:56:04	1
14	Priya	Kapoor	priya@spicekitchen.com	owner_pass123	9444444444	restaurant_owner	2025-10-29 23:56:04	1
15	Carlos	Rodriguez	carlos@tacofiesta.com	owner_pass123	9555555555	restaurant_owner	2025-10-29 23:56:04	1
16	Giovanni	Bianchi	giovanni@pastahouse.com	owner_pass123	9666666666	restaurant_owner	2025-10-29 23:56:04	1
17	Ahmed	Khan	ahmed@bbqnation.com	owner_pass123	9777777777	restaurant_owner	2025-10-29 23:56:04	1

17 rows in set (0.00 sec)

Restaurant menu items-

```
mysql> select * from menu_items;
```

item_id	restaurant_id	name	description	price	image	is_available
1	1	Margherita Pizza	Classic tomato and mozzarella	299.00	NULL	1
2	1	Pepperoni Pizza	Spicy pepperoni with extra cheese	399.00	NULL	1
3	1	Veggie Supreme	Loaded with fresh vegetables	349.00	NULL	1
4	1	Garlic Bread	Crispy bread with garlic butter	99.00	NULL	1
5	2	Classic Burger	Beef patty with lettuce and tomato	249.00	NULL	1
6	2	Chicken Burger	Grilled chicken with special sauce	229.00	NULL	1
7	2	Veggie Burger	Plant-based patty with avocado	199.00	NULL	1
8	2	French Fries	Crispy golden fries	89.00	NULL	1
9	3	California Roll	Crab, avocado, and cucumber	450.00	NULL	1
10	3	Salmon Nigiri	Fresh salmon on rice	550.00	NULL	1
11	3	Vegetable Tempura	Crispy fried vegetables	350.00	NULL	1
12	3	Miso Soup	Traditional Japanese soup	150.00	NULL	1
13	4	Butter Chicken	Creamy tomato curry with chicken	399.00	NULL	1
14	4	Paneer Tikka Masala	Cottage cheese in spicy gravy	349.00	NULL	1
15	4	Garlic Naan	Soft bread with garlic	49.00	NULL	1
16	4	Biryani	Aromatic rice with meat/vegetables	299.00	NULL	1
17	5	Chicken Tacos	Three tacos with salsa	279.00	NULL	1
18	5	Beef Burrito	Large burrito with beans and rice	329.00	NULL	1
19	5	Nachos Supreme	Loaded nachos with cheese	249.00	NULL	1
20	5	Guacamole	Fresh avocado dip	99.00	NULL	1

20 rows in set (0.00 sec)

Drivers table-

```
mysql> select * from drivers;
```

driver_id	user_id	first_name	last_name	phone_num	num_plate	current_location	current_latitude	current_longitude	is_available
1	6	Raj	Kumar	9123456789	MH01AB1234	Andheri, Mumbai	19.11360000	72.86970000	1
2	7	Priya	Sharma	9123456790	MH02CD5678	Bandra, Mumbai	19.05960000	72.82950000	1
3	8	Amit	Patel	9123456791	MH03EF9012	Colaba, Mumbai	18.92200000	72.83470000	0
4	9	Sunita	Gupta	9123456792	MH04GH3456	Dadar, Mumbai	19.01760000	72.84790000	1
5	10	Vikram	Singh	9123456793	MH05IJ7890	Powai, Mumbai	19.11970000	72.90810000	1

5 rows in set (0.00 sec)

Restaurants table-

(didn't print image blob as it's a longblob and would cover the whole screen)

```
mysql> desc restaurants;
```

Field	Type	Null	Key	Default	Extra
restaurant_id	int	NO	PRI	NULL	auto_increment
name	varchar(150)	NO		NULL	
description	text	YES		NULL	
address	varchar(255)	NO		NULL	
latitude	decimal(10,8)	YES		NULL	
longitude	decimal(11,8)	YES		NULL	
phone_num	varchar(20)	YES		NULL	
rating	decimal(2,1)	YES		0.0	
is_open	tinyint(1)	YES		1	
cuisine	varchar(100)	YES		NULL	
image	longblob	YES		NULL	

11 rows in set (0.01 sec)

```
mysql> select restaurant_id,name,description,address,latitude,longitude,is_open,rating,cuisine from restaurants;
```

restaurant_id	name	description	address	latitude	longitude	is_open	rating	cuisine
1	Pizza Place	Best pizzas in town with authentic Italian recipes	100 Food St, Mumbai	19.07600000	72.87770000	1	5.	Italian
2	Burger Hub	Gourmet burgers made with premium ingredients	200 Taste Ave, Mumbai	19.05960000	72.82950000	1	4.	American
3	Sushi World	Fresh sushi and Japanese delicacies	300 Ocean Rd, Mumbai	18.92200000	72.83470000	1	4.	Japanese
4	Spice Kitchen	Authentic Indian curries and tandoor	400 Masala St, Mumbai	19.11360000	72.86970000	1	4.	Indian
5	Taco Fiesta	Mexican street food and tacos	500 Salsa Blvd, Mumbai	19.03300000	73.02970000	1	4.	Mexican
6	Pasta House	Handmade pasta and Italian wines	600 Noodle Lane, Mumbai	19.01760000	72.85620000	0	4.	Italian
7	BBQ Nation	All-you-can-eat BBQ buffet	700 Grill St, Mumbai	19.11970000	72.90810000	1	4.	BBQ

Triggers, Procedures/Functions, Nested query, Join, Aggregate queries-

Refer [https://github.com/pranav-rn/food-delivery-website/blob/main/2 triggers functions procedures.sql](https://github.com/pranav-rn/food-delivery-website/blob/main/2%20triggers%20functions%20procedures.sql)

for triggers, functions and procedures.

Nested queries, joins and aggregate queries used extensively in backend-

```
JS driver-orders.js X
backend > routes > JS driver-orders.js > ...
474 router.get('/closest/:restaurantId', async (req, res) => {
493   const [drivers] = await connection.query(`
494     SELECT
495       d.driver_id,
496       d.num_plate,
497       d.current_location,
498       d.current_latitude,
499       d.current_longitude,
500       u.first_name,
501       u.last_name,
502       u.phone_num,
503       (
504         6371 * acos(
505           cos(radians(?)) *
506           cos(radians(d.current_latitude)) *
507           cos(radians(d.current_longitude) - radians(?)) +
508           sin(radians(?)) *
509           sin(radians(d.current_latitude))
510         )
511       ) AS distance_km,
512       ROUND(
513         (6371 * acos(
514           cos(radians(?)) *
515           cos(radians(d.current_latitude)) *
516           cos(radians(d.current_longitude) - radians(?)) +
517           sin(radians(?)) *
518           sin(radians(d.current_latitude))
519         )) * 3, 0
520       ) AS eta_minutes
521     FROM Drivers d
522     JOIN Users u ON d.user_id = u.user_id
523     WHERE d.is_available = 1
524           AND d.current_latitude IS NOT NULL
525           AND d.current_longitude IS NOT NULL
526     HAVING distance_km <= 50
527     ORDER BY distance_km ASC
528     LIMIT 10`
```

```

JS driver-orders.js X
backend > routes > JS driver-orders.js > ...
179 router.get('/available-orders', authMiddleware, requireRole('driver'), async (req, res) => {
180   try {
181     const connection = db.getConnectionByUserType(req.user.userType);
182
183     // Nested query: Orders where driver_id is NULL or in specific statuses
184     const [orders] = await connection.query(`
185       SELECT
186         o.order_id,
187         o.order_date,
188         o.total_amount,
189         o.address_id,
190         r.name as restaurant_name,
191         r.address as restaurant_address,
192         r.latitude as restaurant_lat,
193         r.longitude as restaurant_lng,
194         a.address,
195         a.city,
196         a.latitude as delivery_lat,
197         a.longitude as delivery_lng
198     FROM Orders o
199     JOIN Restaurants r ON o.restaurant_id = r.restaurant_id
200     LEFT JOIN Addresses a ON o.address_id = a.address_id
201     WHERE o.driver_id IS NULL
202           AND o.order_status = 'confirmed'
203           AND o.order_id NOT IN (
204             SELECT order_id
205             FROM Orders
206             WHERE driver_id IS NOT NULL
207           )
208     ORDER BY o.order_date ASC
209     LIMIT 20
210     `);
211

```

```

// Get items for each order
for (let order of orders) {
  const [items] = await connection.query(`
    SELECT
      oi.quantity,
      oi.price_per_item,
      mi.name as item_name,
      mi.description
    FROM Order_Items oi
    JOIN Menu_Items mi ON oi.item_id = mi.item_id
    WHERE oi.order_id = ?
    `, [order.order_id]);

```

```
// Join query: Get orders with restaurant and customer details
let query = `
    SELECT
        o.order_id,
        o.order_date,
        o.order_status,
        o.total_amount,
        o.address_id,
        o.rating,
        r.restaurant_id,
        r.name as restaurant_name,
        r.address as restaurant_address,
        r.phone_num as restaurant_phone,
        r.latitude as restaurant_latitude,
        r.longitude as restaurant_longitude,
        u.first_name as customer_first_name,
        u.last_name as customer_last_name,
        u.phone_num as customer_phone,
        a.address,
        a.city,
        a.state,
        a.postal_code,
        a.latitude as customer_latitude,
        a.longitude as customer_longitude,
        p.payment_method,
        p.status as payment_status
    FROM Orders o
    JOIN Restaurants r ON o.restaurant_id = r.restaurant_id
    JOIN Users u ON o.user_id = u.user_id
    LEFT JOIN Addresses a ON o.address_id = a.address_id
    LEFT JOIN Payments p ON o.order_id = p.order_id
    WHERE o.driver_id = ?
`;
```

```
// Get driver's current orders
router.get('/:id/orders', async (req, res) => {
  try {
    const [orders] = await db.query(
      `SELECT
        o.*,
        r.name as restaurant_name,
        r.address as restaurant_address,
        a.address as delivery_address,
        a.city,
        a.state,
        u.first_name as customer_first_name,
        u.last_name as customer_last_name,
        u.phone_num as customer_phone
      FROM Orders o
      JOIN Restaurants r ON o.restaurant_id = r.restaurant_id
      JOIN Addresses a ON o.address_id = a.address_id
      JOIN Users u ON o.user_id = u.user_id
      WHERE o.driver_id = ?
      AND o.order_status IN ('assigned', 'picked_up', 'on_the_way')
      ORDER BY o.order_date DESC`,
      [req.params.id]
    );
  }
}
```

```
// Find nearest available driver using Haversine formula
const [drivers] = await db.admin.query(`
  SELECT
    driver_id,
    current_latitude,
    current_longitude,
    (6371 * acos(
      cos(radians(?)) * cos(radians(current_latitude)) *
      cos(radians(current_longitude) - radians(?)) +
      sin(radians(?)) * sin(radians(current_latitude))
    )) AS distance
  FROM Drivers
  WHERE is_available = TRUE
  AND current_latitude IS NOT NULL
  AND current_longitude IS NOT NULL
  ORDER BY distance ASC
  LIMIT 1
`, [restLat, restLng, restLat]);
```



```
const [orders] = await connection.query(
  `SELECT
    o.*,
    r.name as restaurant_name,
    r.address as restaurant_address,
    r.phone_num as restaurant_phone,
    r.latitude as restaurant_latitude,
    r.longitude as restaurant_longitude,
    a.address as delivery_address,
    a.city,
    a.state,
    a.postal_code,
    a.latitude as customer_latitude,
    a.longitude as customer_longitude,
    d.first_name as driver_first_name,
    d.last_name as driver_last_name,
    d.phone_num as driver_phone,
    d.num_plate as driver_plate,
    d.current_latitude as driver_latitude,
    d.current_longitude as driver_longitude,
    u_driver.email as driver_email,
    generate_order_reference(o.order_id) as order_reference
  FROM Orders o
  JOIN Restaurants r ON o.restaurant_id = r.restaurant_id
  JOIN Addresses a ON o.address_id = a.address_id
  LEFT JOIN Drivers d ON o.driver_id = d.driver_id
  LEFT JOIN Users u_driver ON d.user_id = u_driver.user_id
  WHERE o.order_id = ? AND o.user_id = ?`,
  [req.params.id, req.user.userId]
);
```

```
// Aggregate query: Sales by date
const [dailySales] = await connection.query(`
  SELECT
    DATE(o.order_date) as date,
    COUNT(DISTINCT o.order_id) as total_orders,
    SUM(o.total_amount) as total_revenue,
    AVG(o.total_amount) as avg_order_value
  FROM Orders o
  WHERE o.restaurant_id = ?
    AND o.order_status NOT IN ('cancelled')
    AND o.order_date >= DATE_SUB(CURDATE(), INTERVAL 30 DAY)
  GROUP BY DATE(o.order_date)
  ORDER BY date DESC
`, [restaurantId]);
```

```
// Top selling items (nested query with aggregate)
const [topItems] = await connection.query(`
  SELECT
    mi.name as item_name,
    COUNT(oi.item_id) as times_ordered,
    SUM(oi.quantity) as total_quantity,
    SUM(oi.quantity * oi.price_per_item) as total_revenue
  FROM Order_Items oi
  JOIN Menu_Items mi ON oi.item_id = mi.item_id
  WHERE oi.order_id IN (
    SELECT order_id
    FROM Orders
    WHERE restaurant_id = ?
      AND order_status NOT IN ('cancelled')
      AND order_date >= DATE_SUB(CURDATE(), INTERVAL 30 DAY)
  )
  GROUP BY mi.item_id, mi.name
  ORDER BY total_quantity DESC
  LIMIT 10
`);
```

```
// Overall stats
const [stats] = await connection.query(`
  SELECT
    COUNT(*) as total_orders,
    SUM(total_amount) as total_revenue,
    AVG(total_amount) as avg_order_value,
    AVG(rating) as avg_rating
  FROM Orders
  WHERE restaurant_id = ?
    AND order_status NOT IN ('cancelled')
    AND order_date >= DATE_SUB(CURDATE(), INTERVAL 30 DAY)
`, [restaurantId]);
```

```
// Build query with optional status filter
let query = `
  SELECT
    o.order_id,
    o.order_date,
    o.order_status,
    o.total_amount,
    o.rating,
    u.first_name as customer_first_name,
    u.last_name as customer_last_name,
    u.phone_num as customer_phone,
    u.email as customer_email,
    a.address as delivery_address,
    a.city as delivery_city,
    a.latitude as customer_latitude,
    a.longitude as customer_longitude,
    d.first_name as driver_first_name,
    d.last_name as driver_last_name,
    d.phone_num as driver_phone,
    d.num_plate as driver_plate,
    d.current_latitude as driver_latitude,
    d.current_longitude as driver_longitude,
    u_driver.email as driver_email,
    p.payment_method,
    p.status as payment_status
  FROM Orders o
  JOIN Users u ON o.user_id = u.user_id
  JOIN Addresses a ON o.address_id = a.address_id
  LEFT JOIN Drivers d ON o.driver_id = d.driver_id
  LEFT JOIN Users u_driver ON d.user_id = u_driver.user_id
  LEFT JOIN Payments p ON o.order_id = p.order_id
  WHERE o.restaurant_id = ?
`;
;
```

```
// Aggregate earnings query
// Assuming drivers earn 20% commission on total order amount
let query = `
    SELECT
        COUNT(o.order_id) as total_deliveries,
        SUM(o.total_amount * 0.20) as total_earnings,
        AVG(o.total_amount * 0.20) as avg_earning_per_delivery,
        MIN(o.order_date) as first_delivery,
        MAX(o.order_date) as last_delivery
    FROM Orders o
    WHERE o.driver_id = ? AND o.order_status = 'delivered'
`;
```

Code snippets for invoking the Procedures/Functions/Trigger-

```
const [orders] = await connection.query(  
  `SELECT  
    o.*,  
    r.name as restaurant_name,  
    r.address as restaurant_address,  
    r.phone_num as restaurant_phone,  
    r.latitude as restaurant_latitude,  
    r.longitude as restaurant_longitude,  
    a.address as delivery_address,  
    a.city,  
    a.state,  
    a.postal_code,  
    a.latitude as customer_latitude,  
    a.longitude as customer_longitude,  
    d.first_name as driver_first_name,  
    d.last_name as driver_last_name,  
    d.phone_num as driver_phone,  
    d.num_plate as driver_plate,  
    d.current_latitude as driver_latitude,  
    d.current_longitude as driver_longitude,  
    u_driver.email as driver_email,  
    generate_order_reference(o.order_id) as order_reference  
  FROM Orders o  
  JOIN Restaurants r ON o.restaurant_id = r.restaurant_id  
  JOIN Addresses a ON o.address_id = a.address_id  
  LEFT JOIN Drivers d ON o.driver_id = d.driver_id  
  LEFT JOIN Users u_driver ON d.user_id = u_driver.user_id  
  WHERE o.order_id = ? AND o.user_id = ?`,  
  [req.params.id, req.user.userId]  
);
```

```

/
router.post('/', authMiddleware, async (req, res) => {
  try {
    const { restaurantId, addressId, items } = req.body;
    const userId = req.user.userId;
    const connection = db.getConnectionByUserType(req.user.userType || 'customer');

    // Call stored procedure to place order
    const [result] = await connection.query(
      'CALL place_order(?, ?, ?, ?, @order_id)',
      [userId, restaurantId, addressId, JSON.stringify(items)]
    );

    const [orderId] = await connection.query('SELECT @order_id as order_id');
    const newOrderId = orderId[0].order_id;
  }
}

```

```

router.get('/history', authMiddleware, async (req, res) => {
  try {
    const userId = req.user.userId;
    const limit = parseInt(req.query.limit) || 20;
    const connection = db.getConnectionByUserType(req.user.userType || 'customer');

    const [orders] = await connection.query(
      'CALL get_user_order_history(?, ?)',
      [userId, limit]
    );
  }
}

```

```

// Get specific order details
router.get('/:id', authMiddleware, async (req, res) => {
  try {
    const connection = db.getConnectionByUserType(req.user.userType || 'customer');

    const [orders] = await connection.query(
      `SELECT
        o.*,
        r.name as restaurant_name,
        r.address as restaurant_address,
        r.phone_num as restaurant_phone,
        r.latitude as restaurant_latitude,
        r.longitude as restaurant_longitude,
        a.address as delivery_address,
        a.city,
        a.state,
        a.postal_code,
        a.latitude as customer_latitude,
        a.longitude as customer_longitude,
        d.first_name as driver_first_name,
        d.last_name as driver_last_name,
        d.phone_num as driver_phone,
        d.num_plate as driver_plate,
        d.current_latitude as driver_latitude,
        d.current_longitude as driver_longitude,
        u_driver.email as driver_email,
        generate_order_reference(o.order_id) as order_reference
      FROM Orders o
      JOIN Restaurants r ON o.restaurant_id = r.restaurant_id
      JOIN Addresses a ON o.address_id = a.address_id
      LEFT JOIN Drivers d ON o.driver_id = d.driver_id
      LEFT JOIN Users u_driver ON d.user_id = u_driver.user_id
      WHERE o.order_id = ? AND o.user_id = ?`,
      [req.params.id, req.user.userId]
    );
  }
}

```



```
// Calculate order total with tax
router.get('/:id/total-with-tax', async (req, res) => {
  try {
    const taxRate = 0.05; // 5% tax
    const connection = db.admin;

    const [result] = await connection.query(
      'SELECT calculate_order_total_with_tax(?, ?) as total',
      [req.params.id, taxRate]
    );

    res.json({ totalWithTax: result[0].total });
  } catch (error) {
    console.error('Error calculating total:', error);
    res.status(500).json({ error: 'Server error' });
  }
});
```

```
// Estimate delivery time
router.get('/:restaurantId/estimate/:addressId', async (req, res) => {
  try {
    const connection = db.admin;

    const [result] = await connection.query(
      'SELECT estimate_delivery_time(?, ?) as estimated_time',
      [req.params.restaurantId, req.params.addressId]
    );

    res.json({ estimatedTime: result[0].estimated_time });
  } catch (error) {
    console.error('Error estimating delivery time:', error);
    res.status(500).json({ error: 'Server error' });
  }
});
```

```
// Get driver details
router.get('/:id', async (req, res) => {
  try {
    const [drivers] = await db.query(
      `SELECT
        *,
        get_driver_rating(driver_id) as rating
      FROM Drivers
      WHERE driver_id = ?`,
      [req.params.id]
    );
```

```
// Check if restaurant is open now using function
const [openStatus] = await connection.query(
  'SELECT is_restaurant_open_now(?) as is_open',
  [req.params.id]
);

// Check if restaurant is busy using function
const [busyStatus] = await connection.query(
  'SELECT is_restaurant_busy(?) as is_busy',
  [req.params.id]
);
```

```
// Get restaurant revenue (admin endpoint)
router.get('/:id/revenue', async (req, res) => {
  try {
    const connection = db.admin; // Admin operation
    const { startDate, endDate } = req.query;

    const [result] = await connection.query(
      'CALL calculate_restaurant_revenue(?, ?, ?, @total_revenue, @order_count)',
      [req.params.id, startDate, endDate]
    );
```

```

// Only include loyalty functions for customers
let query;
if (req.user.userType === 'customer') {
  query = `SELECT
    user_id,
    first_name,
    last_name,
    email,
    phone_num,
    created_at,
    is_active,
    get_user_loyalty_tier(user_id) as loyalty_tier,
    get_user_avg_order_value(user_id) as avg_order_value
  FROM Users
  WHERE user_id = ?`;

```

```

// Update status
if (status === 'delivered') {
  // Call stored procedure to mark as completed
  await connection.query('CALL complete_order(?, ?)', [orderId, new Date()]);
} else {
  await db.admin.query('UPDATE Orders SET order_status = ? WHERE order_id = ?', [status, orderId]);
}

res.json({ message: 'Order status updated successfully' });
} catch (error) {
  console.error('Update order status error:', error);
  res.status(500).json({ error: 'Server error' });
}
;

```

```

// Get driver details
router.get('/:id', async (req, res) => {
  try {
    const [drivers] = await db.query(
      `SELECT
        *,
        get_driver_rating(driver_id) as rating
      FROM Drivers
      WHERE driver_id = ?`,
      [req.params.id]
    );
  }

```

```

router.get('/history', authMiddleware, async (req, res) => {
  try {
    const userId = req.user.userId;
    const limit = parseInt(req.query.limit) || 20;
    const connection = db.getConnectionByUserType(req.user.userType || 'customer');

    const [orders] = await connection.query(
      'CALL get_user_order_history(?, ?)',
      [userId, limit]
    );

    res.json(orders[0]);
  } catch (error) {
    console.error('Error fetching order history:', error);
    res.status(500).json({ error: 'Server error' });
  }
});

```

```

// Assign driver to order (uses stored procedure)
router.put('/:id/assign-driver', async (req, res) => {
  try {
    const { driverId } = req.body;
    const connection = db.admin; // Admin operation

    await connection.query(
      'CALL assign_driver_to_order(?, ?)',
      [req.params.id, driverId]
    );

    res.json({ message: 'Driver assigned successfully' });
  } catch (error) {
    console.error('Error assigning driver:', error);
    res.status(500).json({ error: error.message || 'Server error' });
  }
});

```

```
// Process refund (uses stored procedure)
router.post('/:id/refund', authMiddleware, async (req, res) => {
  try {
    const { reason } = req.body;
    const connection = db.getConnectionByUserType(req.user.userType || 'customer');

    await connection.query(
      'CALL process_refund(?, ?)',
      [req.params.id, reason]
    );

    res.json({ message: 'Refund processed successfully' });
  } catch (error) {
    console.error('Error processing refund:', error);
    res.status(500).json({ error: 'Server error' });
  }
});
```

```
// Check if restaurant is open now using function
const [openStatus] = await connection.query(
  'SELECT is_restaurant_open_now(?) as is_open',
  [req.params.id]
);

// Check if restaurant is busy using function
const [busyStatus] = await connection.query(
  'SELECT is_restaurant_busy(?) as is_busy',
  [req.params.id]
);
```